

1. Introdução

Esse artefato contém as aulas ministradas para a disciplina Engenharia de Software.

É de suma importância que consultem as referências bibliográficas, e/ou outras fontes de referências como sites, wikis e fórum de discussão para o enriquecimento do aprendizado.

Nos tópicos abaixo, demonstraremos a *timeline* das aulas – guia para orientar os estudos conforme as aulas ministradas data a data – veja também o plano de ensino.

Nos demais tópicos, são demonstrados os assuntos abordados em cada dia das aulas. Observar que alguns temas e conceitos poderão ser ministrados novamente, conforme a capacidade de síntese dos alunos, sendo necessário repetir as aulas passadas.

2. Timeline das aulas

<ainda em construção>

3. O que é processo

É um conjunto de ações e atividades interrelacionadas que são executadas para criar um produto, serviço ou resultado pré-especificado.

Cada processo é caracterizado por suas entradas, ferramentas e técnicas, e suas saídas resultantes.

Exemplo de processos:

- ✓ Realizar pagamento para funcionários
- ✓ Processo de separação de mercadorias para entrega
- ✓ Processo de desenvolvimento de software
- ✓ Fazer um bolo de cenoura com cobertura de chocolate

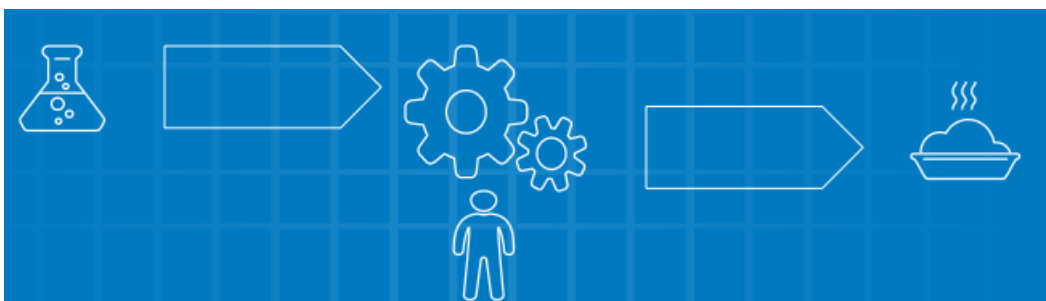


Figura 1 – Processo

Como demonstrado na imagem abaixo, temos as entradas, as técnicas e ferramentas para a execução do mesmo e o resultado esperado – produto do processo.

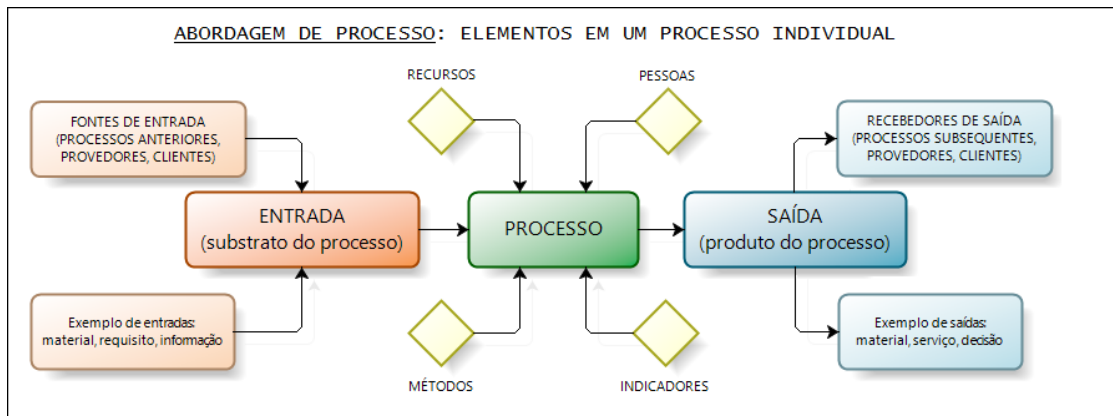


Figura 2 - Processo com suas entradas, execução e sua saída
Fonte: <http://www.8idea.com.br/blog/desvendando-o-mapeamento-de-processos-em-5-minutos/>

4. O que é projeto

É um esforço temporário empreendido para criar um produto, serviço ou resultado exclusivo [PMBok®, 2017].

Variam de tamanho e complexidade, conforme os tipos de produtos ou serviços entregues.

Exemplos de projetos:

- ✓ Construção de uma casa
- ✓ Construção de um protótipo de um avião
- ✓ Construção de uma hidrelétrica
- ✓ Construção de um novo software de gerenciamento de Recursos Humanos

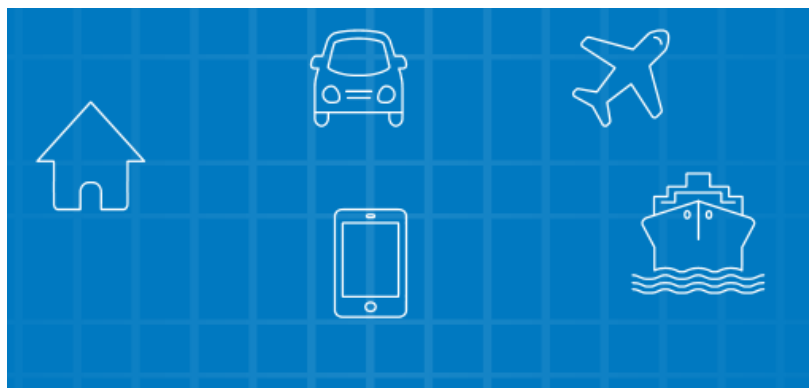


Figura 3 - Projetos

5. O que são boas práticas

Boas práticas consiste em uma(s) técnica(s) identificada(s) e experimentada(s) como eficiente(s) e eficaz(es) em seu contexto de implantação, para a realização de determinada tarefa, atividade ou procedimento ou, ainda, em uma perspectiva mais ampla, para a realização de um conjunto destes, visando o alcance de um objetivo comum.

Em diversas profissões têm sido criadas normas de boas práticas que definem a forma correta de atuar com processos. Como exemplos de tais normas pode-se referir as normas ISO-9001, guia de boas práticas de gestão de projetos do PMI®.

Resumidamente, boa prática significa que existe um acordo geral de que a aplicação dos processos de gerenciamento de projetos pode aumentar as chances de sucesso de projetos.

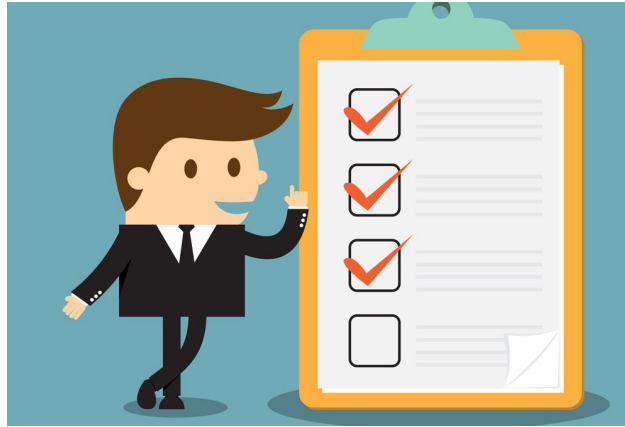


Figura 4 - Boas práticas

6. O que que são metodologias

Metodologia é uma palavra derivada de “método”, do Latim “*methodus*” cujo significado é “caminho ou a via para a realização de algo”. Método é o processo para se atingir um determinado fim ou para se chegar ao conhecimento.

Como é um processo, é obrigatório seguir/executar como foi criado e implantado.

Exemplo de metodologias:

- ✓ RUP – Processo Unificado da Rational - Metodologia de desenvolvimento de software
- ✓ Metodologia de testes de software definidos pela empresa.
- ✓ Metodologia Científica de Pesquisa.



Figura 5 - Metodologia

7. O que é sistema

Um sistema é constituído de um conjunto de elementos (ou componentes) que interagem para um propósito.

Na falta de um componente, o sistema é incapaz de desempenhar as funções que o sistema deve desempenhar com este componente.

Exemplo de sistemas:

- ✓ As peças de um motor formam um sistema; seus componentes isolados nem sequer se parecem com o sistema;
- ✓ Os livros de uma biblioteca, normalmente não formam um sistema; cada um deles isoladamente desempenha perfeitamente sua função; entretanto um livro que é parte de um conjunto de volumes de uma enciclopédia, passa a ser um componente desta enciclopédia.
- ✓ Um sistema de informação, somente com o software propriamente falando, não consegue trabalhar sozinho. Precisa de outros componentes como a interação humana, o hardware para ser executado etc.

Na figura abaixo, são demonstrados os componentes de um SI – Sistema da Informação. Observe que os principais componentes desse sistema compreendem os processos, tecnologia e principalmente as pessoas.

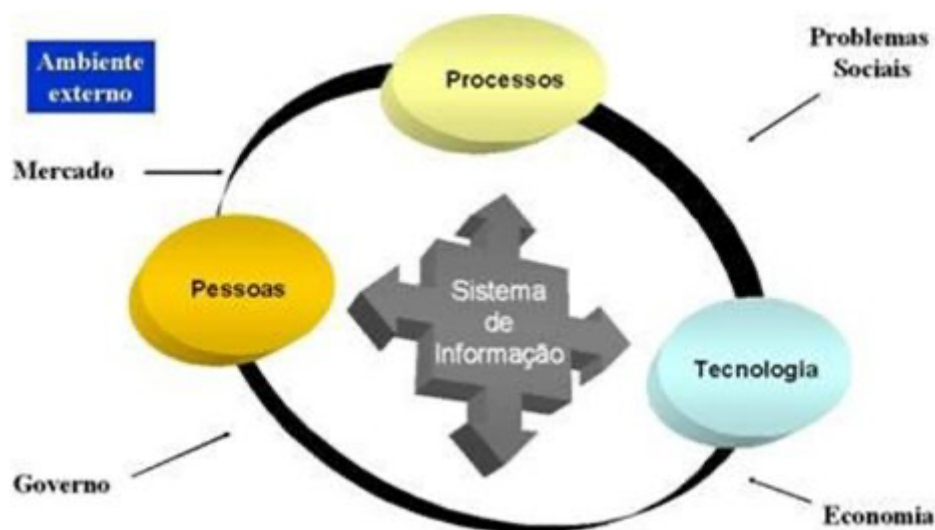


Figura 6 - Exemplo de um sistema de informação e seus componentes
Fonte: <https://queconceito.com.br/sistema>

8. O que é software

Definição clássica: *Software* é uma sentença escrita em uma linguagem computável, para a qual existe uma máquina (computável) capaz de interpretá-la. A sentença (o *software*) é composta por uma sequência de instruções (comandos) e declarações de dados, armazenável em meio digital. Ao interpretar o *software*, a máquina computável é direcionada à realização de tarefas especificamente planejadas, para as quais o *software* foi projetado.

É uma sequência de instruções a serem seguidas e/ou executadas, na manipulação, redirecionamento ou modificação de um dado (informação) ou acontecimento.

“O software é o conjunto de vários artefatos e não apenas o código fonte” (SOMMERVILLE, 2003).



Figura 7 - Software

9. O que é um programa computacional

O termo "Programa" significa o programa original e todas as cópias completas ou parciais do mesmo. Um Programa consiste em instruções legíveis por máquina, seus componentes, dados, conteúdo audiovisual (tal como imagens, texto, gravações ou figuras) e materiais licenciados relacionados.

Um programa consiste:

- ✓ instruções legíveis por máquina,
- ✓ seus componentes,
- ✓ dados,
- ✓ conteúdo audiovisual (tal como imagens, texto, gravações ou figuras) e;
- ✓ materiais licenciados relacionados

9.1. Instruções legíveis por máquina

A definição implica na existência de uma máquina que lê instruções. O termo "instruções" indica a intenção de enviar à máquina algo que esta possa entender, e que tem por meta instruí-la, orientá-la ou comandá-la a realizar algo.

Podemos então inferir que qualquer conjunto de instruções legíveis por máquina, qualquer que seja a máquina, quaisquer que sejam as instruções, podem constituir parte do que chamamos de programa.

Considerando o fato de que toda máquina inventada pelo ser humano possui uma linguagem artificial com a qual nos comunicamos com a máquina, considerando ainda que esta linguagem é composta por cadeias de símbolos que formam sentenças, então qualquer sentença numa linguagem artificial pode constituir parte do que chamamos de programa.

9.2. Seus componentes

Esta é uma parte recursiva da definição. Um programa é composto por componentes, o que implica dizer que um programa pode ser constituído por outros programas menores ou subprogramas.

9.3. Dados

Um programa pode conter dados, o que significa que um programa não é composto apenas por instruções. A existência de dados em um programa está compatível com a definição do Modelo Computacional de Von Neumann, onde um programa armazenado na memória é formado por instruções e dados.

9.4. Conteúdo audiovisual

Este aspecto extrapola o modelo de Modelo Computacional de von Neumann, a medida que torna explícito o fato de que um programa pode ser algo maior do que o que é armazenado na memória no momento de sua execução.

Em outras palavras, qualquer material escrito, impresso, apresentável em qualquer mídia de comunicação, de natureza textual, gráfica, audível, etc, pode pertence ou ser parte de um programa.

9.5. Materiais licenciados relacionados

Do ponto de vista de natureza física dos materiais que compõem um programa, este aspecto veicula a mesma essência de generalidade do item anterior, mas torna explícito o fato de que cada uma das partes do programa está relacionada à existência de produtos ou itens que são criados e pertencem a organizações, e que portanto estão sujeitos às regras de comercialização, distribuição e/ou uso. Para maior aprofundamento, vamos discutir em outro momento sobre Licença de Uso de Software.

10. O que é Qualidade de Software

Vamos começar com esse tema com a imagem abaixo.

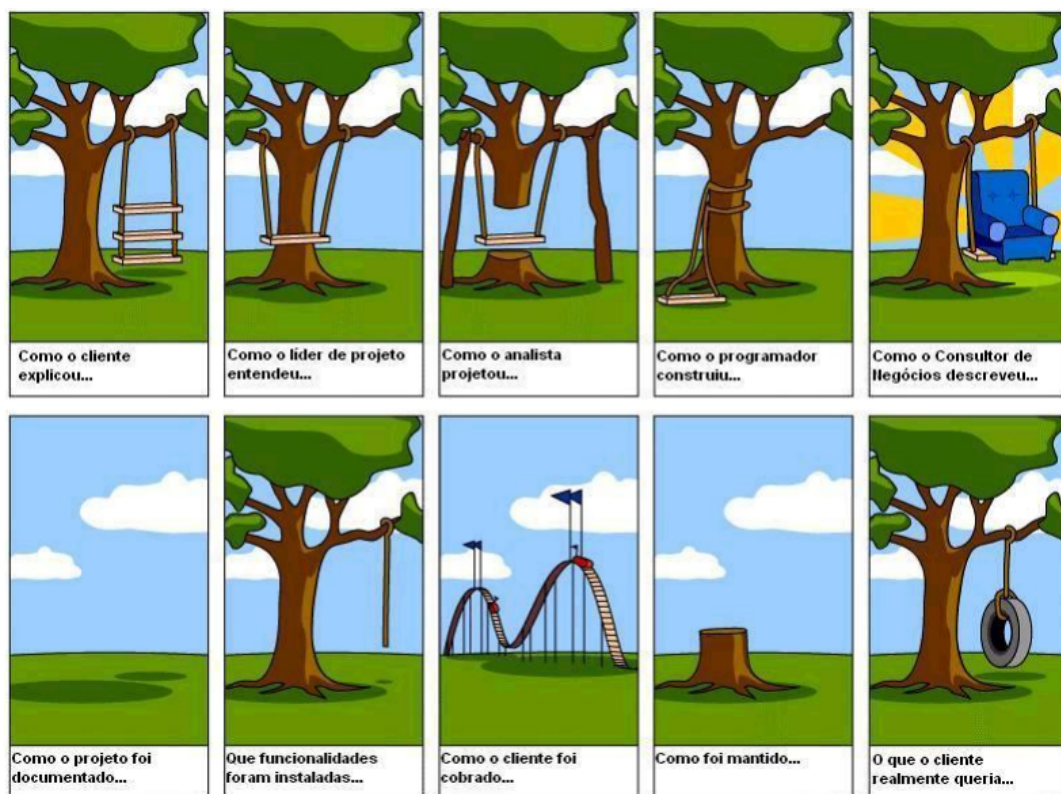


Figura 8 - Qualidade de software

Para Crosby, 1979, "Qualidade é a conformidade às especificações".

Segundo IEEE, define qualidade como “Grau de conformidade de um sistema, componente ou processo com os respectivos requisitos”, ou, alternativamente, como “Grau de conformidade de um sistema, componente ou processo com as necessidades e expectativas de clientes ou usuários”.

Para Ricardo Almeida, é um conceito com múltiplas facetas (perspectivas de usuário, desenvolvedor e cliente) e que envolve diferentes características (por exemplo, usabilidade, confiabilidade, eficiência, manutenibilidade, portabilidade, segurança, produtividade) que devem ser alcançadas em níveis diferentes, dependendo do propósito do software. Por exemplo um software de gestão de recursos humanos de uma empresa é bem diferente de um software que controla uma usina nuclear.

Há o que McCall e Cavano (1978) sugerem como métricas para qualidade de software. Conhecido como Fatores da Qualidade, estes fatores avaliam o *software* em três pontos distintos: Transição do Produto, Revisão do Produto e Operação do Produto. Na Figura abaixo são mostrados os Fatores da Qualidade de McCall:



Figura 9 - Os Fatores da Qualidade de McCall.

Fonte: <http://www.linhadecodigo.com.br/artigo/1712/qualidade-qualidade-de-software-e-garantia-da-qualidade-de-software-sao-as-mesmas-coisas.aspx>

"Qualidade de software é um processo sistemático que focaliza todas as etapas e artefatos produzidos com o objetivo de garantir a conformidade de processos e produtos, prevenindo e eliminando defeitos". (BARTIÉ, 2002, p. 16).

Pressman (2005) sugere que a qualidade de software seja implementada e não somente uma idéia ou desejo que uma organização venha a ter. Para tanto, Pressman (2005) faz as seguintes colocações sobre qualidade de software:

1. "Definir explicitamente o termo qualidade de software, quando o mesmo é dito";(PRESSMAN, 2005, p. 193)
2. "Criar um conjunto de atividades que irão ajudar a garantir que cada produto de trabalho da engenharia de software exiba alta qualidade"; (PRESSMAN, 2005, p. 193)
3. "Realizar atividades de segurança da qualidade em cada projeto de software";(PRESSMAN, 2005, p. 193)
4. "Usar métricas para desenvolver estratégias para a melhoria de processo de software e, como consequência, a qualidade no produto final"; (PRESSMAN, 2005, p. 193)

11. Requisitos funcionais e não funcionais

Entenda-se por requisitos requisito algo que foi requerido, uma condição a ser atendida para satisfazer uma necessidade [PMBOK® 6ª edição].

No que tange a tecnologia da informação, para Sommerville (2008) classifica os requisitos de sistema de software como funcionais, não funcionais e como requisitos de domínio:

- ❖ **Requisitos funcionais** definem as funcionalidades do sistema como deve reagir em condições específicas e como se comportar em determinadas situações. Podem ainda declarar o que o sistema não deve fazer.
- ❖ **Requisitos não funcionais** são restrições sobre serviços ou funções oferecidas pelo sistema. Dentre elas destacam-se restrições de tempo, sobre o processo de desenvolvimento e de padrões. A descrição das restrições complementa a definição de requisitos (PAULA FILHO, 2000).
- ❖ **Requisitos de domínio** são restrições originárias do domínio da aplicação do sistema e refletem características do mesmo. Podem ser requisitos funcionais ou não funcionais.

Resumidamente, temos:

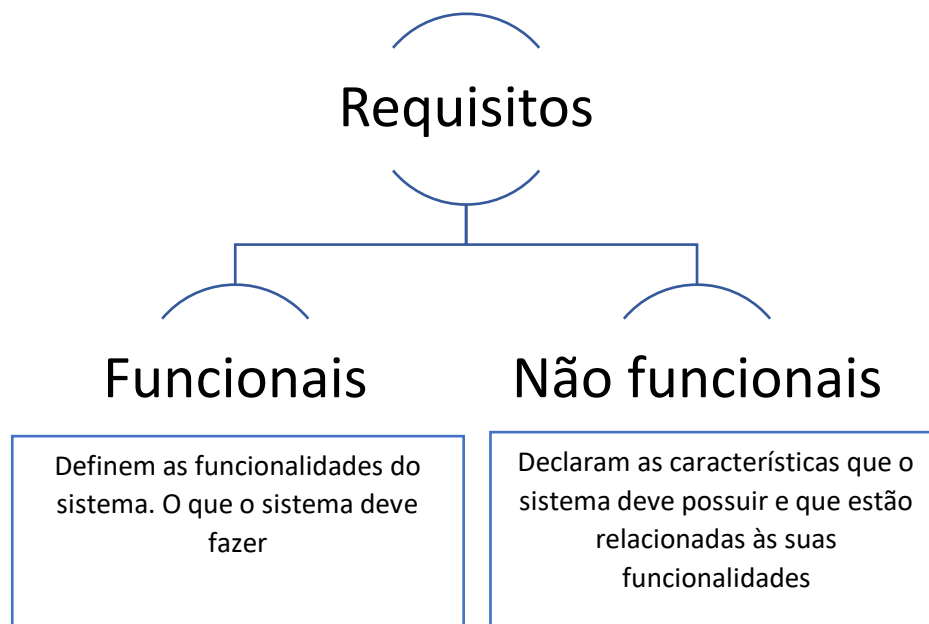


Figura 10 - Requisitos

11.1. Requisitos funcionais

Os requisitos funcionais descrevem a funcionalidade ou os serviços que se espera que o sistema realize em benefício dos usuários (PAULA FILHO, 2000). Eles variam de acordo com o tipo de software em desenvolvimento, com usuários e com o tipo de sistema que está sendo desenvolvido. Requisitos funcionais podem ser expressos de diversas maneiras e, como já foi dito acima, em diferentes níveis de detalhamento. Os requisitos funcionais de usuários definem recursos específicos que devem ser fornecidos pelo sistema (SOMMERVILLE, 2008).

É natural para um desenvolvedor de sistemas interpretar um requisito ambíguo para simplificar sua implementação. Entretanto isso pode não ser necessariamente o que o

cliente necessita. Surgem então novos requisitos e é necessário realizar alterações no sistema, o que afetará diretamente o tempo e o custo do projeto.

É natural para um desenvolvedor de sistemas interpretar um requisito ambíguo para simplificar sua implementação. Entretanto isso pode não ser necessariamente o que o cliente necessita. Surgem então novos requisitos e é necessário realizar alterações no sistema, o que afetará diretamente o tempo e o custo do projeto.

Como exemplo, podemos citar:

- Cadastro de usuários
- Imprimir relatório financeiro
- Cadastrar conta corrente
- Emitir nota fiscal

11.2. Requisitos não funcionais

Os requisitos não funcionais são aqueles que não dizem respeito diretamente às funcionalidades fornecidas pelo sistema. Podem estar relacionados a propriedades de sistemas emergentes, como confiabilidade, tempo de resposta, espaço em disco, desempenho e outros atributos de qualidade do produto (PAULA FILHO, 2000). Às vezes podem dizer respeito ao sistema como um todo. Isso significa que na maioria das vezes eles são mais importantes que os requisitos funcionais individuais. Se uma falha em cumprir um requisito funcional pode comprometer parte do sistema, uma falha em cumprir um requisito não funcional pode tornar todo o sistema inútil (SOMMERVILLE, 2008).

Nem sempre os requisitos não funcionais dizem respeito ao sistema de software a ser desenvolvido. Muitas vezes requisitos não funcionais podem restringir o processo utilizado para desenvolver o sistema. Requisitos de processo podem abranger desde uma especificação de padrões de qualidade a ser utilizada no processo até uma especificação de ferramentas a serem utilizadas.

Os requisitos não funcionais surgem de acordo com as necessidades dos usuários, em razão de restrições orçamentárias, de políticas organizacionais, pela necessidade de interoperabilidade com outros sistemas de software ou hardware e até mesmo em função de fatores externos. Esses últimos podem ser por exemplo de natureza legal ou de segurança.

Sommerville (2008) classifica os requisitos não funcionais em:

- ✓ **Requisitos de produto** que especificam o comportamento do produto. Podem restringir, por exemplo, a liberdade dos projetistas a utilizar uma determinada linguagem.
- ✓ **Requisitos organizacionais** que são procedentes de políticas e procedimentos adotados nas organizações do cliente e do desenvolvedor. Especifica que o sistema deve ser de acordo com um processo-padrão da empresa.
- ✓ **Requisitos externos** que abrange tópicos advindos de fatores externos ao sistema. Dentre eles destacam-se os requisitos de interoperabilidade, os requisitos éticos e os requisitos legais que devem ser observados a fim de garantir que o sistema opera de acordo com a lei.

Um problema dos requisitos não funcionais é que geralmente eles são de difícil verificação e quantificação em uma primeira versão do produto (PAULA FILHO, 2000).

Podem ser escritos para refletir os objetivos gerais do cliente como usabilidade, recuperação à falhas ou rapidez de resposta ao usuário. Esses requisitos são problemáticos na medida em que abrem a possibilidade e múltiplas interpretações e claro discussões quando o sistema é entregue (SOMMERVILLE, 2008).

O ideal é que os requisitos não funcionais sejam expressos de forma quantitativa utilizando-se métricas que possam ser testadas. Algumas das principais métricas para testar os requisitos não funcionais são, a velocidade, o tamanho, a usabilidade, a confiabilidade, a robustez e a portabilidade.

É recomendável que requisitos funcionais e não funcionais sejam diferenciados em um documento de requisitos; embora isso não seja fácil. Se os requisitos não funcionais forem definidos separadamente dos requisitos funcionais será difícil relacioná-los. Por outro lado se forem definidos juntos, será difícil separar considerações funcionais de não funcionais e delinear requisitos que dizem respeito ao sistema como um todo. Sendo assim é preciso encontrar um equilíbrio adequado que irá depender do tipo de sistema. O ideal é que seja criada uma seção separada no documento de requisitos listando aqueles que claramente se relacionam (SOMMERVILLE, 2008).

Como exemplo de requisitos não funcionais, podemos citar:

- Portabilidade da aplicação – acessível em qualquer dispositivo – computador, tablet, celular.
- Segurança – deve-se ser acessado por meio de certificação digital.
- Performance – cada funcionalidade não deve demorar mais que 2 (dois) segundo de resposta ao usuário.
- Configurações mínimas de uso da aplicação – servidores intel xeon, com 64GB de memória RAM.

12. O que é Engenharia de Software;

Engenharia é a aplicação de métodos do conhecimento científico ou empírico destinados à utilização de recursos materiais e naturais para o benefício do ser humano.

É a área do conhecimento que aborda os conceitos de projeção, desenvolvimento, construção, análise e manutenção de alternativas que auxiliem e facilite a vida da sociedade nas atividades cotidianas.

A palavra engenharia tem origem do latim *ingenium*, que significa "produzir ou gerar talento ou qualidade nata". Inicialmente, ela era aplicada a qualquer equipamento mecânico, sobretudo na área militar.

Nos próximos parágrafos, no que tange ao desenvolvimento de desenvolvimento de software, é demonstrado a interpretação de diversos autores.

"Engenharia de software é uma abordagem sistemática e disciplinada para o desenvolvimento de software" (PRESSMAN, 2006).

A Engenharia de Software trata de aspectos relacionados ao estabelecimento de processos, métodos, técnicas, ferramentas e ambientes de suporte ao desenvolvimento e manutenção de software.

Um dos objetivos da Engenharia de Software é melhorar a qualidade dos produtos de softwares desenvolvidos.

"Uma das grandes dificuldades da engenharia do software é resolver o problema e deixar o cliente satisfeito com o software" (JALOTE, 2005).

"A base da engenharia de software são um conjunto de atividades para o processo de desenvolvimento de software. A existência de vários tipos de processo de desenvolvimento de software e podemos dizer para resolver o problema do software usam estas atividades tais como: análise de requisito, design do software, código e teste" (JALOTE, 2005).



Figura 11 - Engenharia de software

13. O que é gerenciamento de projetos de *software*?

Segundo o PMI®, é aplicação do conhecimento, habilidades, ferramentas e técnicas às atividades do projeto para atender aos seus requisitos. No caso de um software, atender os requisitos funcionais e não funcionais do escopo de produto.

O gerenciamento de projetos fornece uma estrutura para gerenciar diversas áreas de conhecimento, segundo a gestão tradicional de projetos e com base no guia PMBOK®, como:

- ✓ Gerenciamento da integração
- ✓ Gerenciamento dos recursos
- ✓ Gerenciamento do escopo
- ✓ Gerenciamento dos custos
- ✓ Gerenciamento da qualidade
- ✓ Gerenciamento do cronograma
- ✓ Gerenciamento dos riscos
- ✓ Gerenciamento das comunicações
- ✓ Gerenciamento das aquisições/fornecedores
- ✓ Gerenciamento das partes interessadas

14. Métodos de Desenvolvimento de *Software* tradicionais e ágeis

Nos tópicos abaixo demonstraremos algumas características de metodologia de desenvolvimento tradicional e conforme a mais recente – manifesto ágil.

Vale ressaltar que a gestão de projetos tradicional e o desenvolvimento ágil definitivamente não são mutuamente excludentes. Enquanto é possível se valer das métricas de gestão, é necessário construir modelos de aferição orientados ao valor agregado nas entregas.

14.1. Método de desenvolvimento Tradicional

Utiliza técnicas, boas práticas e metodologias já consagradas mundialmente, como:

1. Guia de boas práticas de gestão de projetos - PMBOK®
Divide a gestão de projetos em áreas de conhecimento, onde cada uma tem seus processos para auxiliar na entrega de produtos ou serviço.



Figura 12 - PMBOK 6ª edição

Fonte: <https://escritoriodeprojetos.com.br/areas-de-conhecimento-pmbok>

2. RUP - Processo Unificado da *Rational* (em inglês *Rational Unified Process*) é um processo proprietário de Engenharia de software criado pela *Rational Software Corporation*, adquirida pela IBM, ganhando um novo nome IRUP que agora é uma abreviação de IBM *Rational Unified Process* e tornando-se uma *brand* na área de *Software*, fornecendo técnicas a serem seguidas pelos membros da equipe de desenvolvimento de software com o objetivo de aumentar a sua produtividade no processo de desenvolvimento.

É um processo considerado pesado e preferencialmente aplicável a grandes equipes de desenvolvimento e a grandes projetos, porém o fato de ser amplamente customizável torna possível que seja adaptado para projetos de qualquer escala.

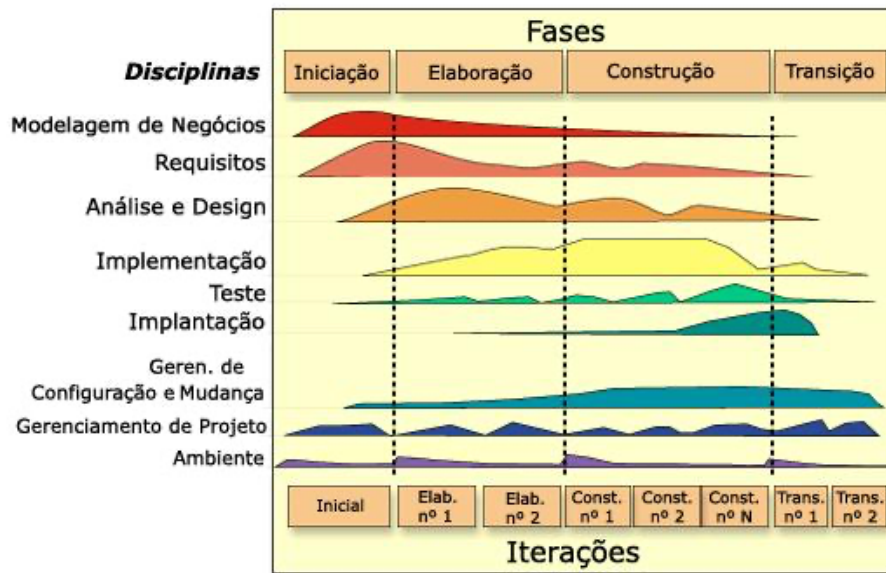


Figura 13 - RUP - Processo Unificado da Rational

14.2. Ágil

O Manifesto Ágil possui a seguinte base:

- ✓ Os indivíduos e as interações são mais importantes do que os processos e as ferramentas;
- ✓ O software funcionando é mais importante do que uma documentação completa;
- ✓ A colaboração com e dos clientes acima de apenas negociações de contratos e;
- ✓ Respostas a mudanças acima de seguir um plano.

A engenharia de *software* ágil combina filosofia com um conjunto de princípios de desenvolvimento. A filosofia defende a satisfação do cliente e a entrega de incremental prévio; equipes de projetos pequenas e altamente motivadas; métodos informais; artefatos de engenharia de software mínimos e, acima de tudo, simplicidade no desenvolvimento geral. Os princípios de desenvolvimento priorizam a entrega mais que a análise e projeto (embora essas atividades não sejam desencorajadas); também priorizam a comunicação ativa e contínua entre desenvolvedores e clientes. (Pressman, 2011).

Nos tópicos abaixo demonstraremos algumas metodologias mais utilizadas atualmente.

14.2.1. Scrum

O Scrum, um framework para desenvolvimento de produtos que apresenta um conjunto de valores, princípios e práticas dentro dos quais pode-se tratar e resolver problemas complexos.

O Scrum tem como base o conhecimento obtido através da experiência e de tomada de decisões baseadas no que é conhecido, sustentando-se em três pilares: transparência, inspeção e adaptação.

No framework Scrum, pequenos times possuem papéis e responsabilidades e, juntos, realizam eventos de duração fixa, que são os ciclos iterativos. Ou seja, um projeto é “quebrado” em projetos menores, com entregas parciais.

SCRUM FRAMEWORK

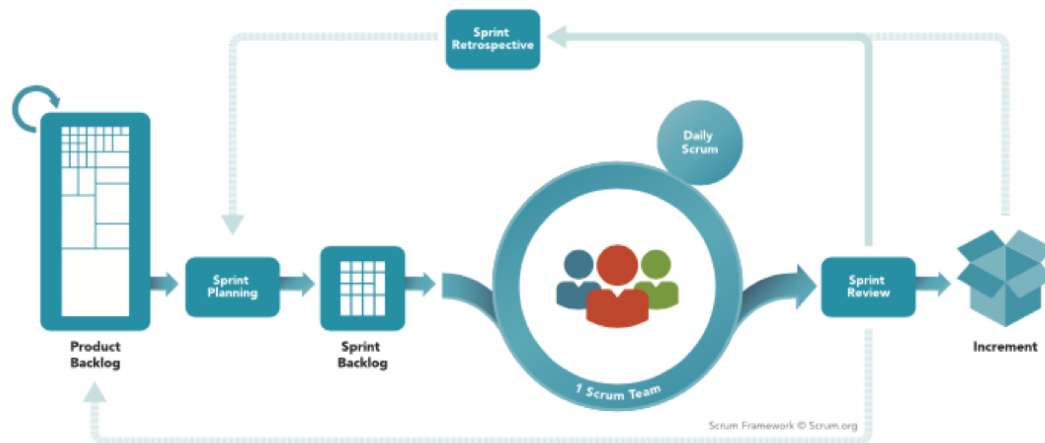


Figura 14 - Framework Scrum

14.2.2. Fundamentos e valores do Scrum

Scrum é um framework leve para gerar valor por meio de soluções adaptativas para problemas complexos. Ele não descreve todas as técnicas de desenvolvimento; estabelece uma estrutura mínima na qual pessoas, eventos e artefatos tornam o trabalho visível e favorecem decisões frequentes com base em evidências.

A abordagem é empírica e se apoia em três pilares: transparência, para que o trabalho e seus critérios sejam compreendidos; inspeção, para verificar regularmente o progresso e os resultados; e adaptação, para ajustar o plano quando a realidade difere do esperado.

Os cinco valores orientam o comportamento do Scrum Team:

- Compromisso: assumir responsabilidade pelo objetivo e pela qualidade do trabalho.
- Foco: concentrar esforços no Sprint Goal e no trabalho mais importante.
- Abertura: tornar visíveis progresso, dificuldades, riscos e aprendizados.
- Respeito: reconhecer as competências, limites e contribuições de cada pessoa.
- Coragem: enfrentar problemas, conversar sobre impedimentos e mudar o que não funciona.

14.2.3. Responsabilidades no Scrum

O Scrum Team reúne Product Owner, Scrum Master e Developers. É uma unidade pequena, multifuncional e autogerenciável, sem subequipes ou hierarquias internas. Todos compartilham a responsabilidade de criar um Increment utilizável e valioso a cada Sprint. O Guia do Scrum usa o termo responsabilidades, embora seja comum encontrar a expressão papéis do Scrum em materiais didáticos.

Responsabilidade	Foco principal	Exemplos de atuação
Product Owner	Maximizar o valor do produto e garantir uma direção clara.	Define e comunica o Product Goal; ordena o Product Backlog; esclarece necessidades; dialoga com usuários e partes interessadas. Pode delegar atividades, mas continua responsável pelas decisões de produto.
Scrum Master	Estabelecer o Scrum e aumentar a efetividade do Scrum Team.	Ensina o framework; facilita quando necessário; ajuda a remover impedimentos; promove colaboração e melhoria contínua; apoia a organização na adoção do Scrum.
Developers	Planejar e produzir um Increment que atenda à Definition of Done.	Selecionam como realizar o trabalho; criam o Sprint Backlog; adaptam o plano diariamente; cuidam da qualidade. Incluem todas as competências necessárias, não apenas programação.

14.2.4. Eventos do Scrum

Os eventos, também chamados informalmente de ritos ou cerimônias, criam uma cadência de inspeção e adaptação. A Sprint contém todos os demais eventos. As durações máximas abaixo consideram uma Sprint de um mês; em Sprints menores, os eventos normalmente são mais curtos.

Evento	Finalidade	Participantes e duração máxima
Sprint	Transformar ideias em valor e avançar em direção ao Product Goal. Tem duração fixa de até um mês; uma nova Sprint começa imediatamente após a anterior.	Todo o Scrum Team. Até um mês.
Sprint Planning	Definir por que a Sprint é valiosa, o que pode ser realizado e como o trabalho selecionado será feito. O resultado inclui o Sprint Goal e o Sprint Backlog.	Scrum Team. Até 8 horas.
Daily Scrum	Inspecionar o progresso em direção ao Sprint Goal e adaptar o plano de trabalho. Não é reunião de prestação de contas ao gestor.	Developers. 15 minutos por dia.
Sprint Review	Inspecionar o resultado com partes interessadas, discutir mudanças no contexto e adaptar o Product Backlog. É uma sessão de trabalho, não apenas uma demonstração.	Scrum Team e partes interessadas convidadas. Até 4 horas.
Sprint Retrospective	Planejar melhorias na qualidade e na efetividade do time, considerando pessoas, interações, processos, ferramentas e Definition of Done.	Scrum Team. Até 3 horas.

14.2.5. Artefatos e compromissos

Cada artefato torna uma parte essencial do trabalho transparente e possui um compromisso que orienta a inspeção. Backlogs não são documentos congelados: evoluem conforme o time aprende sobre produto, usuários, tecnologia e mercado.

Artefato	O que representa	Compromisso associado
Product Backlog	Lista ordenada e emergente do que é necessário para melhorar o produto. É a única fonte de trabalho realizado pelo Scrum Team.	Product Goal: descreve um estado futuro do produto e dá direção de longo prazo.
Sprint Backlog	Sprint Goal, itens selecionados do Product Backlog e plano acionável criado pelos Developers para entregar o Increment.	Sprint Goal: objetivo único que dá coerência à Sprint e permite flexibilidade sobre o trabalho exato.
Increment	Resultado concreto e utilizável que se soma aos incrementos anteriores. Pode haver vários increments durante uma Sprint.	Definition of Done: entendimento compartilhado das condições de qualidade necessárias para considerar o trabalho concluído.

Refinamento do Product Backlog: atividade contínua de decompor e detalhar itens, acrescentando descrição, ordem e tamanho. O refinamento ajuda na preparação de Sprints futuras, mas não é um evento formal do Scrum.

14.2.6. Exemplo prático de Scrum

Cenário: a Faculdade UniBras deseja melhorar seu portal acadêmico. Alunos precisam consultar notas, receber avisos e solicitar documentos. O Product Goal é permitir que o aluno resolva as principais demandas acadêmicas pelo celular, com menos atendimento presencial.

Exemplo de Product Backlog inicialmente ordenado:

Ordem	Item do Product Backlog	Valor esperado	Critério resumido de aceitação
1	Como aluno, quero entrar com minha matrícula para acessar meus dados acadêmicos.	Acesso seguro ao portal.	Credenciais válidas permitem acesso; tentativa inválida informa o erro sem expor dados.
2	Como aluno, quero consultar minhas notas por disciplina para acompanhar meu desempenho.	Autonomia e redução de atendimentos.	Exibir período, disciplina, avaliações, notas e situação.
3	Como aluno, quero receber avisos acadêmicos para não perder prazos.	Melhor comunicação.	Listar avisos por data e marcar os ainda não lidos.
4	Como aluno, quero solicitar declaração de matrícula para obter o documento sem ir à secretaria.	Agilidade no serviço acadêmico.	Permitir solicitar, acompanhar status e baixar o documento aprovado.

Sprint de duas semanas: o Product Owner propõe como Sprint Goal permitir que o aluno acesse o portal e acompanhe suas notas. Durante a Sprint Planning, os Developers selecionam os dois primeiros itens e criam um plano técnico com autenticação, integração com dados acadêmicos, interface responsiva e testes.

No quarto dia, a Daily Scrum revela um impedimento: o serviço legado de notas responde lentamente. Os Developers ajustam o plano, criam dados simulados para continuar a interface e solicitam apoio para a integração. O Scrum Master ajuda a aproximar o time responsável pelo serviço, sem assumir o trabalho técnico dos Developers.

Na Sprint Review, alunos convidados testam o Increment e pedem que a situação final da disciplina seja mais visível. O Product Owner registra e ordena essa necessidade no Product Backlog. Na Sprint Retrospective, o time percebe que integrou tarde demais e decide validar dependências externas já nos primeiros dias da próxima Sprint.

Exemplo de Definition of Done: código revisado; testes automatizados essenciais aprovados; requisitos de segurança verificados; interface testada em celular e computador; documentação técnica atualizada; funcionalidade integrada e disponível no ambiente definido pelo time.

14.2.7. Erros comuns na adoção do Scrum

- Tratar o Product Owner como alguém que apenas escreve requisitos, sem autoridade real para ordenar o produto.
- Transformar o Scrum Master em chefe da equipe, secretário de reuniões ou distribuidor de tarefas.
- Usar a Daily Scrum como relatório individual de status, em vez de inspeção e adaptação do plano pelos Developers.
- Considerar a Sprint Review apenas uma apresentação, sem colaboração nem adaptação do Product Backlog.
- Encerrar itens sem atender à Definition of Done ou criar uma fase de testes separada depois da Sprint.
- Alterar continuamente o Sprint Goal; mudanças podem ocorrer, mas não devem colocar o objetivo da Sprint em risco.

Referência recomendada: SCHWABER, Ken; SUTHERLAND, Jeff. O Guia do Scrum: o guia definitivo para o Scrum - as regras do jogo. Versão oficial vigente, novembro de 2020. Disponível em: <https://scrumguides.org/download.html>.

14.2.8. *Extreme Programming (XP)*

O XP é um método de desenvolvimento de software, leve, não é prescritivo, e procura fundamentar as suas práticas por um conjunto de valores:

- ✓ Comunicação
- ✓ Simplicidade
- ✓ Feedback
- ✓ Coragem
- ✓ Coach

O objetivo principal do XP é levar ao extremo um conjunto de práticas que são ditas como boas na engenharia de software. Entre elas podemos citar o teste.

O XP muda o paradigma, onde não temos o medo da mudança, pois o errar é feito com um baixo custo. Diferente do tradicional em que se diz que quanto mais tarde a mudança, maiores são os custos, e assim sendo nunca devemos fazer mudanças o XP diz que devemos sim estar constantemente fazendo mudanças e não devemos teme-las, principalmente quando seguimos os seus valores e as suas práticas.

Glossário

Modularização	Em tecnologia da informação é um conceito onde o sistema ou software é dividido em partes distintas. Compõe o ferramental necessário para um programa mais legível com uma melhor manutenção e melhor desempenho por meio da programação estruturada.
Terminologia	Conjunto de termos específicos ou sistema de palavras us. numa disciplina particular (p.ex., a terminologia da botânica, da marinharia, da matemática); nomenclatura.

Referências bibliográficas da disciplina

Bibliografia Básica:

1. BRAUDE, Eric. Projetos de software: da programação à arquitetura. Porto Alegre: Bookman, 2005. 619 p.
2. BLAHA, Michael e RUMBAUGH, James. Modelagem e projetos baseados em objetos com UML 2. ed. Rio de Janeiro: Campus, 2006. 496 p.
3. BRAGA, Rosana T. Vaccare e LARMAN, Craig. Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento. 3. ed. Porto Alegre: Bookman, 2007. 695 p.

Bibliografia Complementar:

1. PRESSMAN, Roger S. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: Bookman, 2016. 940 p.
2. TONSIG, Sérgio Luiz. Engenharia de software - Análise e Projeto de Sistemas. 2. ed. Rio de Janeiro: Ciência Moderna, 2013. 319 p.
3. MELNIKOFF, Selma Shin Shimizu e SOMMERVILLE, Ian. Engenharia de software. 8. ed. São Paulo: Pearson Addison, 2007. 552 p.
4. MAGELA, Rogério. Engenharia de software aplicada: fundamentos. Rio de Janeiro: Alta Books. 418 p.
5. MAGELA, Rogério. Engenharia de software aplicada: princípios. Rio de Janeiro: Alta Books. 336 p.
6. COUTO, Ana Brasil. CMMI: integração dos modelos de capacitação e maturidade de sistemas. Rio de Janeiro: Ciência Moderna, 2007.

Referências bibliográficas extras:

1. SOMMERVILLE, Ian. Engenharia de Software: 8 ed. Rio de Janeiro: Prentice-Hall, 2008.
2. PRESSMAN, Roger S. Engenharia de Software: 6 ed. São Paulo: McGraw Hill/Nacional, 2006.
3. PMBOK. Um guia do conhecimento em gerenciamento de projetos (Guia PMBOK). 4 ed. Pennsylvania: Project Management Institute, Inc., 2008.
4. PAULA FILHO, Wilson de Pádua. Engenharia de Software: fundamentos, métodos e padrões. São Paulo: LTC Editora, 2000.
5. Acessado em 09/09/2019, Qualidade de software,
<http://www.linhadecodigo.com.br/artigo/1712/qualidade-qualidade-de-software-e-garantia-da-qualidade-de-software-sao-as-mesmas-coisas.aspx#ixzz5yziPtNfO>

ANEXOS

1. Modelo padrão para requisitos não funcionais

Lista de requisitos <u>Não</u> Funcionais		
Categoria:		
Autor:	Revisão:	Data Atualização:
Código	Nome	Descrição

2. Modelo padrão para requisitos funcionais

Lista de requisitos Funcionais		
Autor:	Revisão:	Data Atualização:
Código	Nome	Descrição

3. Modelo padrão para levantamento das partes interessadas

Lista de partes interessadas		
Autor:	Revisão:	Data Atualização:
Nome	Função	Descrição

Disciplina: Fundamentos da Engenharia de Software

Prof. Esp., PMP: Otávio Cardoso do Espírito Santo Neto

E-mail:

otavio.neto@ueg.br

otavio.cardoso.neto@gmail.com

“Boas coisas acontecem quando você se dedica ao estudo!”